

Al analizar con MetaInfo el archivo de audio AnalisisTextos.mp3 pudimos extraer los siguientes metadatos:

- * Formato: MPEG Audio (mp3)
- * Duración: 7.12 segundos
- * Tasa de bits: 256 kb/s (constante)
- * Canales: Mono - 1 canal
- * Frecuencia de muestreo: 48.000Hz (48kHz)

General	
Complete name	: AnalisisTextos.mp3
Format	: MPEG Audio
File size	: 219 KiB
Duration	: 7 s 12 ms
Overall bit rate mode	: Constant
Overall bit rate	: 256 kb/s
Writing library	: LAME ^L
Audio	
Format	: MPEG Audio
Format version	: Version 1
Format profile	: Layer 3
Duration	: 7 s 12 ms
Bit rate mode	: Constant
Bit rate	: 256 kb/s
Channel(s)	: 1 channel
Sampling rate	: 48.0 kHz
Frame rate	: 41.667 FPS (1152 SPF)
Compression mode	: Lossy
Stream size	: 219 KiB (100%) / 219 KiB (100%) / 219 KiB (100%) / 219 KiB (100%)
Writing library	: LAME ^L

Para resamplear el audio .mp3 y convertirlo en .wav, usamos ffmpeg en la consola.

```
Simbolo del sistema
C:\Users\Ayelen>ffmpeg -i "C:\Users\Ayelen\Desktop\Analisis audios\AnalisisTextos.mp3" -ar 16000 -ac 1 "C:\Users\Ayelen\Desktop\Analisis audios\AnalisisTextos2.wav"
ffmpeg version 7.1.1-essentials_build-www.gyan.dev Copyright (c) 2000-2025 the FFmpeg developers
  built with gcc 14.2.0 (Rev1, Built by MSYS2 project)
  configuration: --enable-gpl --enable-version3 --enable-static --disable-w32threads --disable-autodetect --enable-fontconfig --enable-iconv --enable-gnutls --enable-libxml2 --enable-gmp --enable-bzlib --enable-lzma --enable-zlib --enable-libsrt --enable-libssh --enable-libzmq --enable-avisynth --enable-sdl2 --enable-libwebp --enable-libx264 --enable-libx265 --enable-libxvid --enable-libaom --enable-libopenjpeg --enable-libvpx --enable-mediaserver --enable-libass --enable-libfreetype --enable-libfribidi --enable-libharfbuzz --enable-libvidstab --enable-libvmaf --enable-libzimg --enable-amf --enable-cuda-llvm --enable-cuvid --enable-dxva2 --enable-d3d11va --enable-d3d12va --enable-ffnvcodec --enable-libvpl --enable-nvdec --enable-nvenc --enable-vaapi --enable-libgme --enable-libopenmpt --enable-libopencore-amrwb --enable-libmp3lame --enable-libtheora --enable-libvo-amrwbenc --enable-libgsm --enable-libopencore-amrnb --enable-libopus --enable-libspeex --enable-libvorbis --enable-librubberband
  libavutil      59. 39.100 / 59. 39.100
  libavcodec     61. 19.101 / 61. 19.101
  libavformat    61.  7.100 / 61.  7.100
  libavdevice    61.  3.100 / 61.  3.100
  libavfilter    10.  4.100 / 10.  4.100
  libswscale     8.  3.100 /  8.  3.100
  libswresample  5.  3.100 /  5.  3.100
  libpostproc   58.  3.100 / 58.  3.100
[mp3 @ 000002214be21540] Estimating duration from bitrate, this may be inaccurate
Input #0, mp3, from 'C:\Users\Ayelen\Desktop\Analisis audios\AnalisisTextos.mp3':
  Duration: 00:00:07.01, start: 0.000000, bitrate: 256 kb/s
  Stream #0:0: Audio: mp3 (mp3float), 48000 Hz, mono, fltp, 256 kb/s
Stream mapping:
  Stream #0:0 -> #0:0 (mp3 (mp3float) -> pcm_s16le (native))
Press [q] to stop, [?] for help
Output #0, wav, to 'C:\Users\Ayelen\Desktop\Analisis audios\AnalisisTextos2.wav':
  Metadata:
    ISFT           : Lavf61.7.100
  Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 16000 Hz, mono, s16, 256 kb/s
  Metadata:
    encoder        : Lavc61.19.101 pcm_s16le
[mp3float @ 000002214be34bc0] invalid new backstep -1
[out#0/wav @ 000002214be2a180] video:0KiB audio:220KiB subtitle:0KiB other streams:0KiB global headers:0KiB muxing overhead: 0.034663%
size=      220KiB time=00:00:07.03 bitrate= 256.1kbits/s speed= 380x
```

Tal como se aprecia en la imagen, en la línea de comando le damos los parámetros para el resampleo. En este caso queremos que tenga 16.000Hz de frecuencia de muestreo y que el canal fuera 1 (mono), y obtenemos un nuevo archivo de audio llamado "AnálisisTextos2.wav", que es con el que vamos a trabajar en python.

Una vez generado nuestro nuevo audio en formato wav, lo analizamos con MediaInfo para ver los metadatos que tiene este nuevo audio y ver si coincide con los parámetros que le pedimos a la consola que tuviera.

```
General
Complete name      : AnalisisTextos2.wav
Format             : Wave
Format settings    : PcmWaveformat
File size          : 220 KiB
Duration           : 7 s 32 ms
Overall bit rate mode : Constant
Overall bit rate   : 256 kb/s
Writing application : Lavf61.7.100

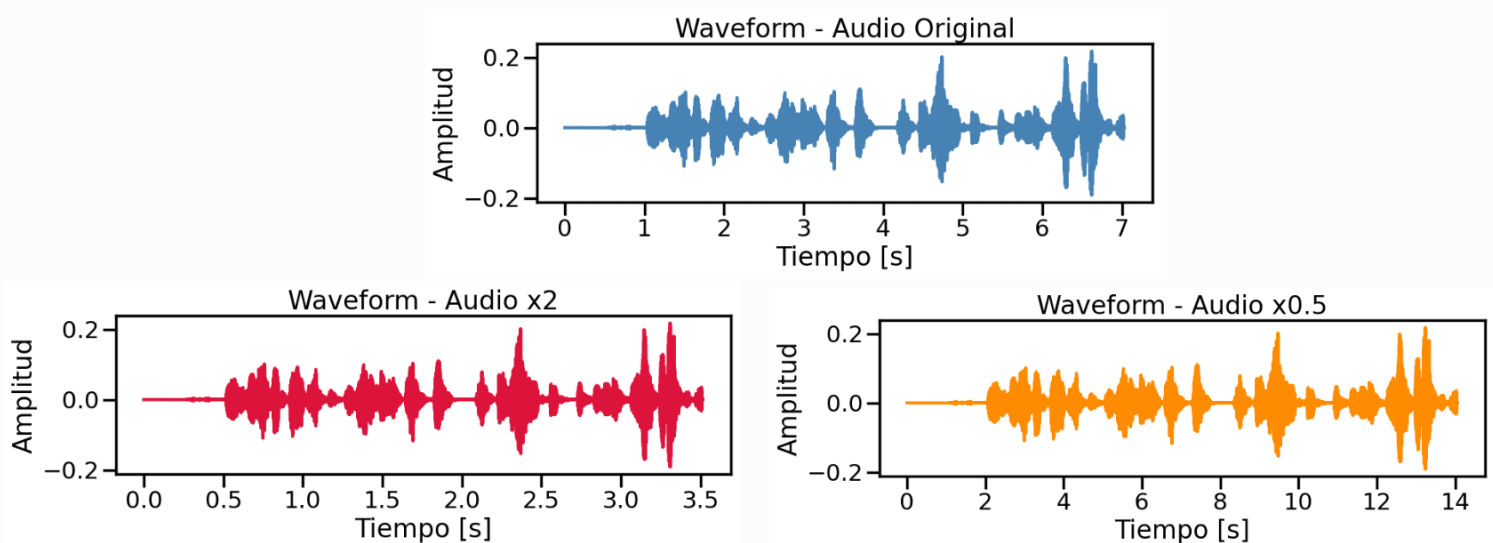
Audio
Format             : PCM
Format settings    : Little / Signed
Codec ID           : 1
Duration           : 7 s 32 ms
Bit rate mode      : Constant
Bit rate           : 256 kb/s
Channel(s)         : 1 channel
Sampling rate      : 16.0 kHz
Bit depth          : 16 bits
Stream size        : 220 KiB (100%) / 220 KiB (100%)
```

Del nuevo archivo de audio AnalisisTextos2.wav pudimos extraer los siguientes metadatos:

- * Formato: Wave (wav)
- * Duración: 7.32 segundos
- * Tasa de bits: 256 kb/s (constante)
- * Canales: Mono - 1 canal
- * Frecuencia de muestreo: 16.000Hz (16kHz)

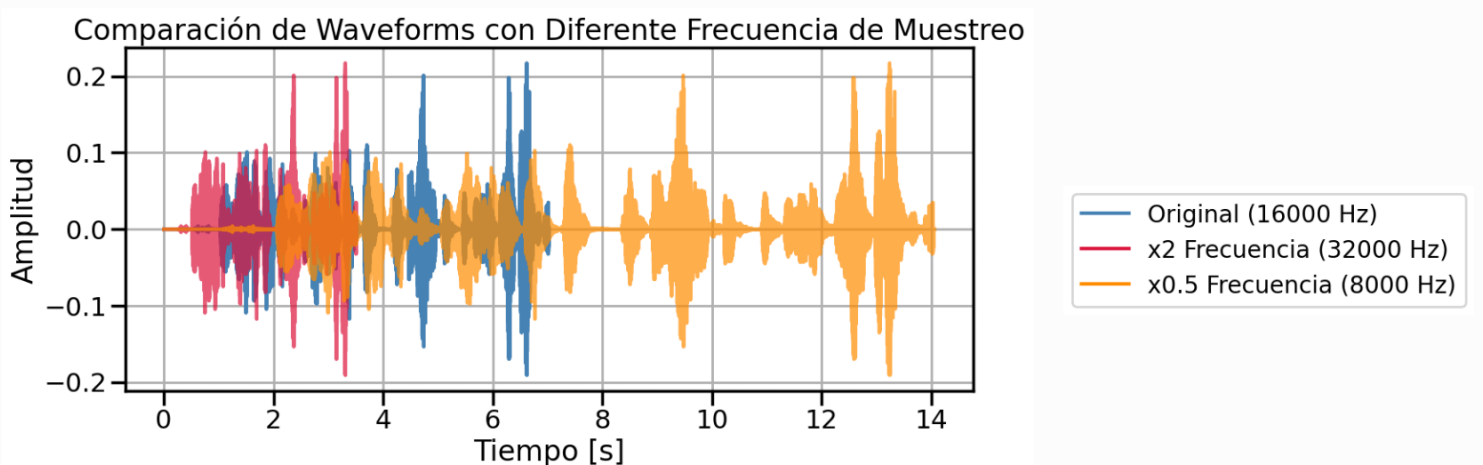
En el archivo "analisis.py" hemos realizado el análisis del archivo de audio en formato .wav, para observar qué es lo que ocurre al efectuar modificaciones en la frecuencia de muestreo y de la calidad de audio del mismo.

Para cada cambio de frecuencia de muestreo generamos un gráfico de forma de onda (waveform), en donde se muestra la evolución de la amplitud del audio a lo largo del tiempo.



Tal como podemos observar, la amplitud de las ondas de todas las muestras son iguales. Si duplicamos la frecuencia de muestreo las ondas se compactan, mientras que si la reducimos a la mitad las ondas se estiran.

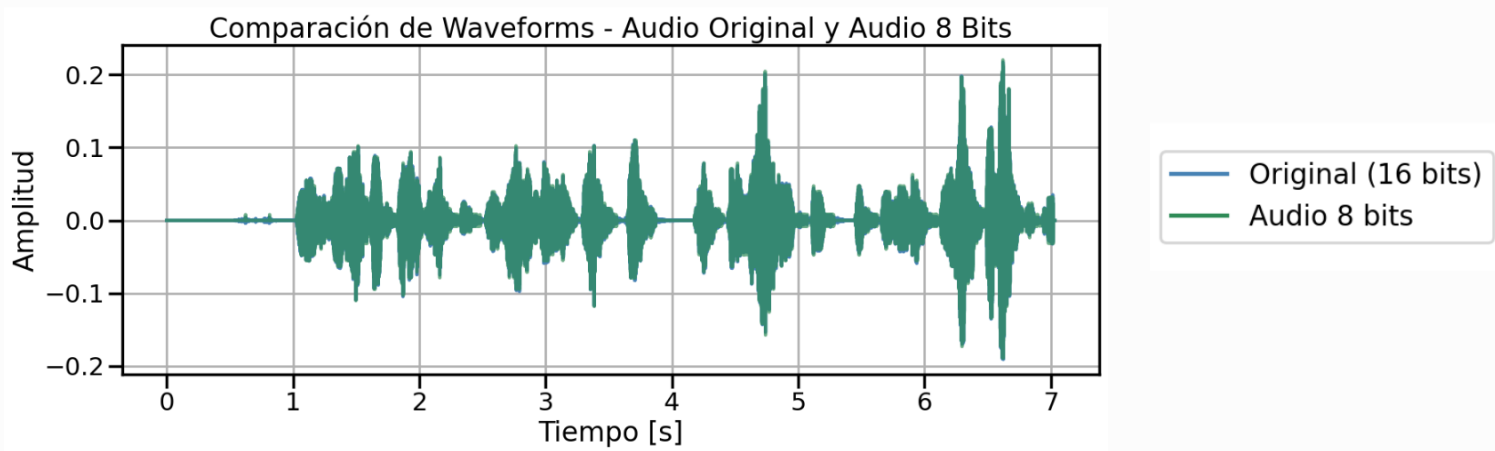
En el siguiente gráfico vamos a poder observar esa diferencia:



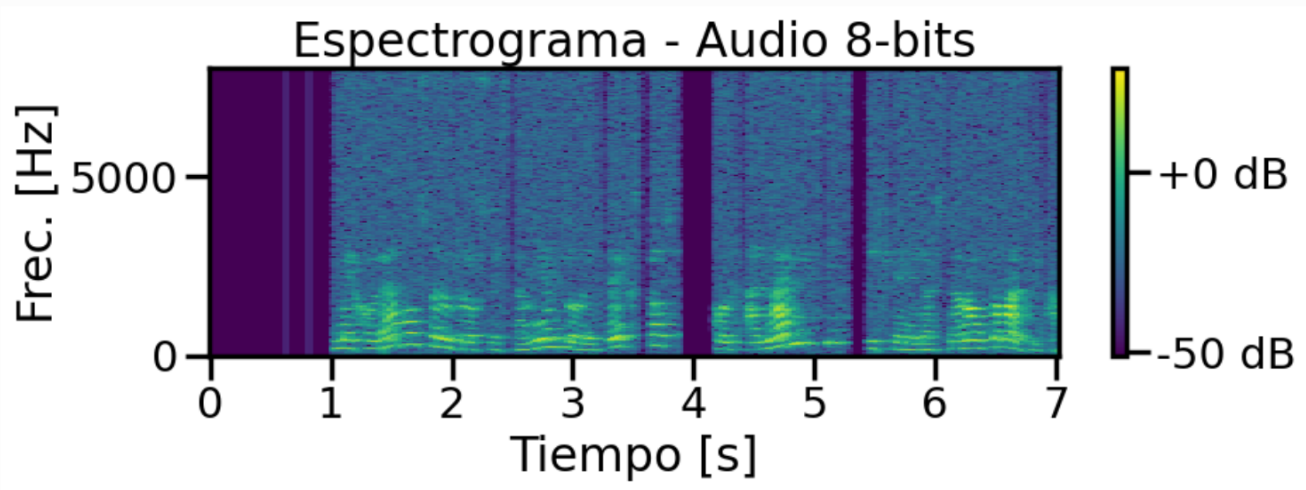
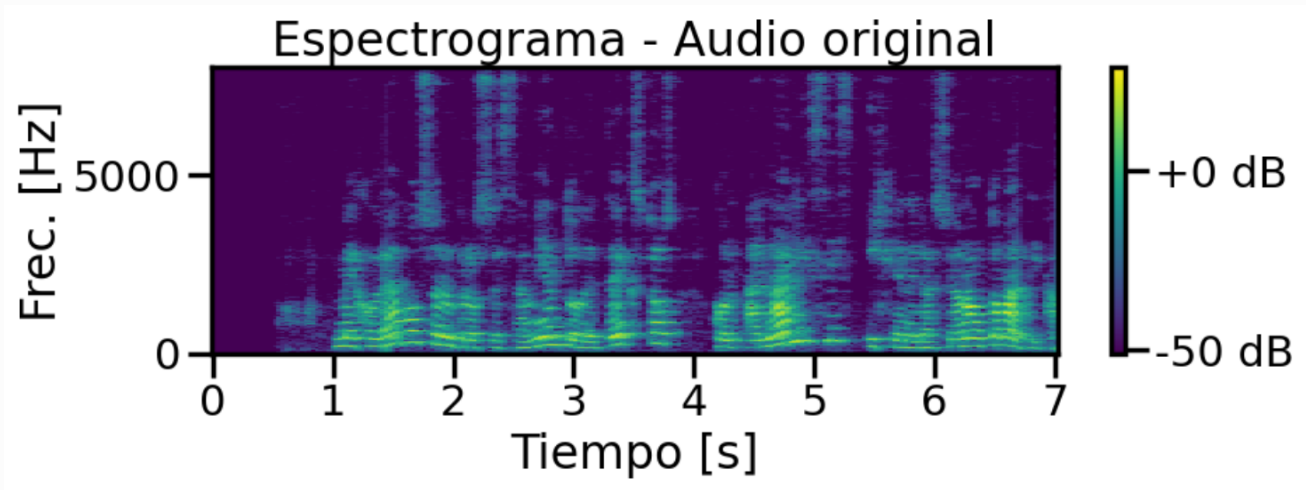
Al aumentar la frecuencia de muestreo la duración del audio disminuye (en este caso, se reduce a la mitad) y el sonido es más agudo; mientras que si reducimos la frecuencia de muestreo la duración del audio aumenta (en nuestro caso, se incrementa el doble) y el sonido se vuelve más grave.

Otra modificación que podemos hacer a través de Python es en la calidad del archivo de audio. Nuestro audio original con el que trabajamos, estaba en formato .wav y en calidad de 16 bits. Si a esa

calidad del audio la reducimos a 8 bits, podemos notar que hay mayor ruido de fondo (se escucha en el fondo un ruido arenoso).



En nuestros gráficos en forma de onda no podemos notar ninguna diferencia y, si las hay, son casi imperceptibles. Por eso optamos por realizar otro tipo de gráfico en el que si podemos ver diferencias entre ambos audios. Usamos el gráfico de espectrograma con el cual podemos observar que efectivamente cambiaron los audios y el de baja calidad se nota con más ruido que el original.



Sin embargo, si realizamos los espectrogramas de los audios de frecuencia de muestreo reducida y aumentada, no tendrían ninguna diferencia notable con respecto al del audio original.

** Al ejecutar el código de Python, se va a generar una copia en su dispositivo de los archivos de audio con la modificación de frecuencia de muestreo y con la modificación de calidad, para que pueda escuchar las diferencias que hay entre cada uno.

Así mismo, se va a guardar un archivo .txt con el vector completo que se generó a partir del audio original, ya que en el editor solo se muestra una mínima parte de él por ser muy largo.